UniCesumar
EDUCAÇÃO A DISTÂNCIA

Protocolo de Finalização Nº 000101432763

O aluno João Guilherme Tito de Jesus com RA **24098553-5**
finalizou a atividade **ATIVIDADE 1 - ADSIS - ESTRUTURAS, PESQUISA E ORDENAÇÃO DE DADOS - 51_2026**
em **25/02/2026 13:25:13**

ATIVIDADE 1 - ADSIS - ESTRUTURAS, PESQUISA E ORDENAÇÃO DE DADOS - 51_2026

Período:23/02/2026 08:00 a 22/03/2026 23:59 (Horário de Brasília)
Status:ABERTO
Nota máxima:0,50 **Nota obtida:**

1ª QUESTÃO

Uma **rede de farmácias** possui diversas filiais, cada uma com sua própria base de dados de clientes. Para unificar esses dados, foi criada uma tabela chamada **Clientes** com **N registros**, possivelmente contendo clientes repetidos, pois o mesmo cliente pode visitar diferentes filiais. A rede deseja consolidar esses dados em uma tabela **Clientes_Unicos**, contendo apenas os clientes únicos.

Para isso, foi proposto o seguinte processo:
Etapa 1 – Ordenação dos dados:
A tabela Clientes será ordenada usando o algoritmo Merge Sort para facilitar a detecção de duplicatas.

Etapa 2 – Eliminação de duplicatas:
Após a ordenação, a tabela Clientes será percorrida. Cada cliente será comparado com o anterior, e apenas os clientes únicos serão copiados para a tabela Clientes_Unicos.

Essas etapas estão representadas no código a seguir:

```c
void merge(int *v, int ini, int meio, int fim) {
    int n1 = meio - ini + 1;
    int n2 = fim - meio;

    int L[n1], R[n2];  // Subarrays temporários

    // Copia os dados para os subarrays L[] e R[]
    for (int i = 0; i < n1; i++)
        L[i] = v[ini + i];
    for (int j = 0; j < n2; j++)
        R[j] = v[meio + 1 + j];

    int i = 0, j = 0, k = ini;

    // Intercala os subarrays
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {  // Preserva estabilidade
            v[k] = L[i];
            i++;
        } else {
            v[k] = R[j];
            j++;
        }
        k++;
    }

    // Copia os elementos restantes de L[], se houver
    while (i < n1) {
        v[k] = L[i];
        i++;
        k++;
    }

    // Copia os elementos restantes de R[], se houver
    while (j < n2) {
        v[k] = R[j];
        j++;
        k++;
    }
}
```

Com base nas etapas descritas e no código, responda às seguintes perguntas:
a) Analise a complexidade computacional de cada etapa do processo (ordenar a tabela Clientes com Merge Sort e percorrer para eliminar duplicatas). Justifique sua resposta.

b) Considere que o algoritmo Merge Sort utiliza memória extra para os subarrays temporários em cada divisão. Qual seria a quantidade total de memória adicional utilizada durante a execução, em função de N? Explique o cálculo.

c) Explique como o algoritmo Merge Sort garante a estabilidade na ordenação. Por que essa propriedade é importante para resolver o problema descrito?

Orientações gerais:

- Acesse o link com um vídeo tutorial para ajudá-lo nesse processo de criação e desenvolvimento. O acesso deverá ser realizado em: Materiais >> Material da Disciplina.
- Lembre-se que evidências de cópias de materiais, incluindo de outros acadêmicos, sem devidas referências, serão inquestionavelmente zeradas. As citações e referências, mesmo que do livro da disciplina, devem ser realizadas conforme normas da Instituição de Ensino.
- Não são permitidas correções parciais no decorrer do módulo, ou seja, o famoso: “professor, veja se minha atividade está certa?”. Isso invalida seu processo avaliativo. Lembre-se que a interpretação da atividade também faz parte da avaliação.
- Procure sanar suas dúvidas junto à mediação em tempo hábil sobre o conteúdo exigido na atividade, de modo que consiga realizar sua participação.
- Sua resposta deve ser inserida diretamente no quadro disponível no campo da Atividade de Estudo 1 (não é permitido anexar formulário).
- Procure elaborar sua resposta em um editor de textos, existem vários grátis, que funcionam diretamente no navegador, como, por exemplo, o Google Docs e depois colar no campo da atividade.
- Atenção ao prazo de entrega, evite envio de atividade em cima do prazo. Você pode ter algum problema com internet, computador, software etc., e os prazos não serão flexibilizados, mesmo em caso de comprovação.

ALTERNATIVAS

a) Análise da Complexidade Computacional
Etapa 1 (Merge Sort): Complexidade O(nlogn) em todos os casos. Isso ocorre pois o algoritmo utiliza a estratégia de divisão e conquista, dividindo a lista repetidamente até chegar a sublistas unitárias para depois intercalá-las de forma ordenada.
Etapa 2 (Eliminação de duplicatas): Complexidade O(n). Uma vez que os dados estão ordenados, registros idênticos ficam adjacentes, permitindo que uma única varredura linear identifique e remova as duplicatas.
b) Quantidade de Memória Adicional